ATIVIDADE · FASE 1

Rabisco

Especificação de uma linguagem para desenhar: do texto à AST

Laboratório de Programação Avançada II · Compiladores

Integrantes: Vitor Utimura Locateli (RA 23024638) e Lara Marina de Oliveira (RA 23024708)

Data: 21/10/2026

1. Descrição da linguagem

Rabisco é uma linguagem simples para descrever desenhos 2D. O programa começa dizendo o tamanho da tela e depois lista as formas (retas, círculos e retângulos) e os grupos. Um grupo junta formas e outros grupos, e pode ter grupo dentro de grupo sem limite. O grupo também pode ser girado e deslocado (são os modificadores). Nesta fase o programa só é analisado: o resultado final é a árvore sintática abstrata (AST), que vai virar SVG na Fase 2.

1.1 Elementos

Elemento	Sintaxe	Observação
Tela	`tela LARGURA ALTURA`	Obrigatória, uma única vez, no início.
Reta	`reta X1 Y1 X2 Y2 [estilo]`	Dois pontos: início e fim.
Círculo	`circulo CX CY RAIO [estilo]`	Centro e raio.
Retângulo	`retangulo X Y LARGURA ALTURA [estilo]`	Canto de cima à esquerda, largura e altura.
Estilo	`borda=COR fundo=COR espessura=N`	Os três são opcionais, cada um no máximo uma vez, nesta ordem.
Grupo	`grupo [modificadores] { ... }`	Pelo menos um elemento dentro das chaves.
Modificadores	`girar ANGULO mover DX DY`	Zero ou mais, em qualquer ordem, podem repetir.
Comentário	`% texto até o fim da linha`	Ignorado pelo analisador léxico.
Números	`0`, `25`, `400`	Inteiros sem sinal, sem zero à esquerda.
Cores	`azul`, `#87CEEB`	10 nomes ou #RRGGBB.

1.2 Exemplo 1: carro

O corpo é um retângulo; cada roda é um grupo deslocado até a posição certa. A primeira roda contém um grupo interno, girado, com um raio.

```
tela 400 300
% um carro simples
retangulo 50 150 200 60 fundo=azul
grupo mover 70 190 {
    circulo 0 0 25 fundo=preto
    grupo girar 45 {
        reta 0 0 20 0 borda=branco espessura=2
    }
}
grupo mover 200 190 {
    circulo 0 0 25 fundo=preto
}
```

1.3 Exemplo 2: casa com sol

Usa cores hexadecimais, os três atributos de estilo juntos, comentários no fim da linha e um grupo com dois modificadores (`mover` e depois `girar`).

```
tela 300 300
% casa com sol
retangulo 0 0 300 300 fundo=#87CEEB        % céu
retangulo 0 220 300 80 fundo=verde         % grama
```

```
grupo mover 80 120 {
    retangulo 0 40 120 100 borda=preto fundo=amarelo espessura=2
    reta 0 40 60 0 borda=vermelho espessura=3
    reta 60 0 120 40 borda=vermelho espessura=3
    retangulo 45 90 30 50 fundo=#8B4513
}
grupo mover 240 60 girar 15 {
    circulo 0 0 30 fundo=laranja
    grupo girar 45 {
        reta 0 0 50 0 borda=amarelo
    }
}
```

2. Definição léxica

O léxico testa as regras na ordem da tabela, sempre na posição atual do texto (flag y do JavaScript), e fica valendo a primeira que der certo. `\b` quer dizer fim de palavra: a palavra-chave só vale se depois dela não vier letra (com ou sem acento), número ou _.

Token	Expressão regular	Descrição
ESPACO	`\s+`	Espaços, tabulações e quebras de linha. **Ignorado.**
COMENTARIO	`%[^\n]*`	Do % até o fim da linha. **Ignorado.**
TELA	`tela\b`	Palavra reservada: declaração da tela
RETA	`reta\b`	Palavra reservada: forma reta
CIRCULO	`circulo\b`	Palavra reservada: forma círculo
RETANGULO	`retangulo\b`	Palavra reservada: forma retângulo
GRUPO	`grupo\b`	Palavra reservada: início de grupo
GIRAR	`girar\b`	Modificador de grupo: rotação
MOVER	`mover\b`	Modificador de grupo: deslocamento
BORDA	`borda\b`	Atributo de estilo: cor do traço
FUNDO	`fundo\b`	Atributo de estilo: cor do preenchimento
ESPESSURA	`espessura\b`	Atributo de estilo: espessura do traço
COR	`(preto|branco|vermelho|verde|azul|amarelo|laranja|roxo|cinza|rosa)\b`	Cor por nome (lista fechada de 10)
HEX	`#[0-9A-Fa-f]{6}\b`	Cor hexadecimal #RRGGBB
NUM	`(0|[1-9][0-9]*)\b`	Inteiro sem sinal, incluindo o zero, sem zero à esquerda
IGUAL	`=`	Liga o atributo de estilo ao seu valor
ABRE	`\{`	Abre o bloco do grupo
FECHA	`\}`	Fecha o bloco do grupo

Tokens ignorados: ESPACO e COMENTARIO. O léxico reconhece esses trechos e pula eles, mas não coloca na lista de tokens.

Erro léxico: acontece quando nenhuma regra serve na posição atual. O léxico guarda a linha e o trecho errado (a palavra inteira, até o próximo espaço), pula esse trecho e continua. Assim todos os erros léxicos do programa aparecem de uma vez.

2.1 Por que \b nas palavras-chave e nos números

- Sem `\b`, a regra `reta` casaria com o começo de `retangulo` (e `retangulos` viraria RETA + resto). Com `\b`, `retangulo` só é RETANGULO, e `retangulos` é um erro léxico inteiro. A regra RETANGULO também vem antes de RETA, por segurança.
- Sem `\b`, `azulado` viraria COR `azul` + erro `ado`, e `05` viraria NUM `0` + NUM `5`. Com `\b`, esses trechos são erros léxicos completos.
- O comentário usa `%`, que não conflita com o `#` das cores hexadecimais.

3. Definição sintática

Notação das aulas: palavras em MAIÚSCULAS são tokens (terminais), palavras em minúsculas são variáveis, `|` separa alternativas e `ε` representa a cadeia vazia. **Símbolo inicial: programa.**

```
programa      → TELA NUM NUM elementos
elementos     → elemento elementos | ε
elemento      → forma | grupo
forma         → RETA NUM NUM NUM NUM estilo
              | CIRCULO NUM NUM NUM estilo
              | RETANGULO NUM NUM NUM NUM estilo
estilo        → borda fundo espessura
borda         → BORDA IGUAL cor | ε
fundo         → FUNDO IGUAL cor | ε
espessura     → ESPESSURA IGUAL NUM | ε
cor           → COR | HEX
grupo         → GRUPO modificadores ABRE elemento elementos FECHA
modificadores → modificador modificadores | ε
modificador   → GIRAR NUM | MOVER NUM NUM
```

3.1 Por que a gramática funciona no parser descendente recursivo

- **Sem recursão à esquerda:** nenhuma regra começa pela própria variável. As listas (`elementos` e `modificadores`) usam recursão à direita.
- **Escolha pelo próximo token:** em cada variável que tem mais de uma opção, cada opção começa com tokens diferentes, então o parser não tem como confundir. Quando tem `ε`, ela só é escolhida quando o próximo token é um dos que podem vir depois da variável (conjunto SEGUINTE), e esses tokens também não aparecem nas outras opções. A tabela abaixo mostra como o parser decide.
- No código, cada variável virou uma função, cada token virou um `esperar()`, cada `|` virou um `if`, e as listas viraram `while` (que fazem a mesma coisa que a recursão à direita da gramática).

Variável	Alternativa	Escolhida quando o próximo token é
elementos	elemento elementos	RETA, CIRCULO, RETANGULO, GRUPO
	ε	FECHA ou fim do programa ($)
elemento	forma	RETA, CIRCULO, RETANGULO
	grupo	GRUPO

Variável	Alternativa	Escolhida quando o próximo token é
forma	RETA … / CIRCULO … / RETANGULO …	RETA / CIRCULO / RETANGULO
borda	BORDA IGUAL cor	BORDA
	ε	FUNDO, ESPESSURA, RETA, CIRCULO, RETANGULO, GRUPO, FECHA, $
fundo	FUNDO IGUAL cor	FUNDO
	ε	ESPESSURA, RETA, CIRCULO, RETANGULO, GRUPO, FECHA, $
espessura	ESPESSURA IGUAL NUM	ESPESSURA
	ε	RETA, CIRCULO, RETANGULO, GRUPO, FECHA, $
cor	COR / HEX	COR / HEX
modificadores	modificador modificadores	GIRAR, MOVER
	ε	ABRE
modificador	GIRAR NUM / MOVER NUM NUM	GIRAR / MOVER

Erro sintático: o parser para no primeiro token que não encaixa e mostra a linha dele, o que era esperado e o que veio no lugar (tipo e lexema). Se o programa acabar antes da hora, aparece “o fim do programa” e a linha é a do último token. Exemplo:

```
Erro sintático na linha 5: esperado RETA, CIRCULO, RETANGULO ou GRUPO, encontrado FECHA ("}")
```

4. Árvore sintática abstrata (AST)

A AST é gerada só quando não há erros. Os números ficam guardados como número (Number, não texto), e não tem pontuação nem delimitadores (=, {, } somem), mantém um grupo dentro do outro do mesmo jeito que está no programa e guarda os modificadores na ordem em que foram escritos.

Nó (campo tipo)	Campos	O que representa
programa	tela: nó tela elementos: lista de formas e grupos	O programa inteiro. elementos pode ser vazio.
tela	largura: número altura: número	Tamanho do desenho.
reta	x1, y1, x2, y2: números estilo: objeto estilo	Segmento de (x1, y1) até (x2, y2).
circulo	cx, cy, raio: números estilo: objeto estilo	Círculo de centro (cx, cy).
retangulo	x, y, largura, altura: números estilo: objeto estilo	Retângulo com canto em (x, y).
(estilo)	borda?: texto da cor fundo?: texto da cor espessura?: número	Objeto sem campo tipo. Só contém os atributos escritos; sem atributos é {}.
grupo	modificadores: lista de girar/mover elementos: lista (mín. 1)	Bloco de formas e grupos; a lista de modificadores segue a ordem do texto.

Nó (campo tipo)	Campos	O que representa
girar	angulo: número	Modificador de rotação.
mover	dx: número dy: número	Modificador de deslocamento horizontal e vertical.

4.1 AST do Exemplo 1 (carro)

Visão em árvore (a mesma que a página de teste mostra na aba Árvore):

```
programa
├─ tela 400 x 300
├─ retangulo (50, 150) 200 x 60  {fundo=azul}
├─ grupo  [mover 70 190]
│  ├─ circulo centro (0, 0) raio 25  {fundo=preto}
│  └─ grupo  [girar 45]
│     └─ reta (0, 0) → (20, 0)  {borda=branco, espessura=2}
└─ grupo  [mover 200 190]
   └─ circulo centro (0, 0) raio 25  {fundo=preto}
```

AST em JSON, exatamente como gerada pelo parser (objetos curtos numa linha só, para caber na página):

```
{
  "tipo": "programa",
  "tela": {"tipo": "tela", "largura": 400, "altura": 300},
  "elementos": [
    {
      "tipo": "retangulo",
      "x": 50,
      "y": 150,
      "largura": 200,
      "altura": 60,
      "estilo": {"fundo": "azul"}
    },
    {
      "tipo": "grupo",
      "modificadores": [{"tipo": "mover", "dx": 70, "dy": 190}],
      "elementos": [
        {
          "tipo": "circulo",
          "cx": 0,
          "cy": 0,
          "raio": 25,
          "estilo": {"fundo": "preto"}
        },
        {
          "tipo": "grupo",
          "modificadores": [{"tipo": "girar", "angulo": 45}],
          "elementos": [
            {
              "tipo": "reta",
              "x1": 0,
              "y1": 0,
              "x2": 20,
              "y2": 0,
              "estilo": {"borda": "branco", "espessura": 2}
            }
          ]
        }
      ]
    },
    {
      "tipo": "grupo",
      "modificadores": [{"tipo": "mover", "dx": 200, "dy": 190}],
```

```
      "elementos": [
        {
          "tipo": "circulo",
          "cx": 0,
          "cy": 0,
          "raio": 25,
          "estilo": {"fundo": "preto"}
        }
      ]
    }
  ]
}
```

5. Decisões de projeto

5.1 Léxico

- **Palavras-chave** em minúsculas e sem acento (`circulo`, `retangulo`), como `avancar` no tutorial. A linguagem diferencia maiúsculas: `Reta` é erro léxico.
- **Prefixos:** `reta` e `retangulo` começam igual. As regras usam `\b`, então `retangulo` nunca é quebrado em `reta` + `ngulo`, e uma palavra maior como `retangulos` é erro léxico.
- **Zero:** `0` é aceito. **Zero à esquerda** (`05`, `007`) é erro léxico, e o trecho inteiro é informado. Número colado em letras (`12abc`) também é erro léxico.
- **Comentário:** `%` até o fim da linha, em qualquer lugar (inclusive antes da tela e no fim de uma linha com código). Não conflita com `#`, e um `#FF0000` dentro do comentário é ignorado junto com ele.
- **Cores:** 10 nomes (preto, branco, vermelho, verde, azul, amarelo, laranja, roxo, cinza, rosa) e `#RRGGBB` com exatamente 6 dígitos hexadecimais, maiúsculos ou minúsculos. `#FFF` (3 dígitos) e `#12G45F` são erros léxicos. A cor vai para a AST exatamente como foi escrita.
- **Espaços:** podem ser usados à vontade entre os tokens. `fundo=azul` e `fundo = azul` dão no mesmo, e dá até para escrever o programa todo em uma linha só.
- **Vários erros léxicos:** todos são listados (linha e trecho). Se houver qualquer um, a análise sintática não roda.

5.2 Sintaxe

- **Tela:** obrigatória, primeiro token do programa e uma única vez. Uma segunda `tela` é erro sintático.
- **Programa sem formas** (só a tela) é válido: gera uma tela em branco.
- **Grupo vazio** é erro sintático. A regra `grupo → GRUPO modificadores ABRE elemento elementos FECHA` exige pelo menos um elemento.
- **Atributos de estilo:** opcionais, cada um **no máximo uma vez**, em ordem fixa `borda`, `fundo`, `espessura`. Atributo repetido ou fora de ordem é erro sintático. Motivo: assim a gramática fica simples, continua dando para decidir olhando só o próximo token, e a AST nunca fica com dois valores para o mesmo atributo.
- **Modificadores:** só em grupos, entre `grupo` e `{`; zero ou mais, em qualquer ordem, **podem repetir**. Motivo: um modificador soma com o outro (`girar 30 girar 30` equivale a 60°) e a ordem muda o resultado, por isso a AST guarda a lista na ordem escrita.
- **Estilo só em formas e modificadores só em grupos.** Um grupo não tem estilo próprio; uma forma não tem modificadores.

- **Valores estranhos** (`tela 0 0`, raio 0, `espessura=0`, ângulo 720) são aceitos: o sintático só confere se o programa está escrito do jeito certo. Se o valor faz sentido ou não, fica para a Fase 2.

5.3 Erros e AST

- **Erro sintático:** o parser para no primeiro erro. A mensagem mostra a linha do token, o que era esperado e o que veio no lugar (tipo e lexema). Em alguns erros comuns (atributo fora de ordem, modificador fora de grupo, segunda tela) a mensagem também mostra uma dica, mas isso não muda o que é aceito.
- **Atributos ausentes** não aparecem na AST (o objeto `estilo` só tem o que foi escrito). Os valores padrão serão decididos na Fase 2.
- **Tela como nó próprio** (`programa.tela`), separada da lista `elementos`, já que ela não é uma forma.

6. Implementação e testes

Arquivo	Conteúdo
`index.html`	Página de teste: caixa de texto, botão Analisar, menu com os casos de teste.
`main.js`	Liga a página aos analisadores; mostra tokens (linha, tipo, lexema), erros ou a AST (JSON e árvore).
`tokens.js`	Lista de regras {tipo, padrao, ignorar} com as expressões regulares da seção 2. No código o \b foi escrito como (?![\p{L}\d_]), porque o \b do JavaScript não entende letra com acento (retaé virava RETA + erro).
`lexer.js`	Analisador léxico: ponteiro, regex sticky, contagem de linhas, erros léxicos.
`parser.js`	Analisador descendente recursivo: uma função por variável da gramática; gera a AST.
`casos.js / casos-de-teste.md`	Os 8 casos de teste com o resultado esperado.
`testes.mjs`	Confere os 8 casos automaticamente: node testes.mjs.

Sem bibliotecas de análise. Para abrir a página é preciso um servidor local (os módulos não carregam com duplo clique): Live Server no VS Code ou `python -m http.server 8000`.

6.1 Casos de teste

Caso	O que testa	Resultado esperado
V1	Carro (grupo dentro de grupo)	Válido; AST gerada
V2	Casa com sol (hexadecimal, vários modificadores)	Válido; AST gerada
V3	Tela em branco (só tela e comentários)	Válido; AST gerada
V4	Três níveis de grupo e modificadores repetidos	Válido; AST gerada
V5	Paleta completa, zeros e formas sem estilo	Válido; AST gerada
I1	Erros léxicos (zero à esquerda, cor inexistente, palavra-chave colada, hex inválido)	linha 2: "05" linha 2: "azulado" linha 3: "retangulos" linha 3: "#12G45F"

Caso	O que testa	Resultado esperado
I2	Erro sintático: grupo vazio	Linha 5: esperado RETA, CIRCULO, RETANGULO ou GRUPO, encontrado FECHA ("}")
I3	Erro sintático: atributos de estilo fora de ordem	Linha 2: esperado RETA, CIRCULO, RETANGULO, GRUPO ou o fim do programa, encontrado BORDA ("borda")

O código de cada caso e a AST esperada completa estão em `casos-de-teste.md`.